Final Report

Inter-IIT Tech. Meet 14.0

Pathway

Real-Time Streaming AI System for Fraud Detection and Customer Targeting.

Submitted By:
Team - 82

Contents

1. Introduction

Modern banking systems operate under strict latency, security, and regulatory constraints while processing high-volume, high-velocity data streams. Traditional batch-oriented and siloed banking pipelines struggle to address real-time threats such as *credit card frauds* [3] and rapidly changing customer financial intent. Periodic feature refreshes and offline model retraining result in stale risk assessments, delayed detection, and suboptimal customer targeting, leading to financial losses and reduced customer trust.

This project presents a **unified, streaming-native AI platform** built on Pathway that operates entirely on live banking event streams. The system integrates *credit card fraud detection* and *customer targeting* within a single low-latency architecture. By continuously ingesting transactions, login events, and customer profile updates, the platform enables in-flight risk scoring, live feature maintenance, incremental model updates, and explainable decision reporting. We demonstrate the system through a real-time targeted calling pipeline for *pre-approved car loans* and a streaming credit card fraud detection pipeline. The same architecture extends naturally to other products such as *ELSS and equity investments*, personal loans, and home loans without architectural changes.

2. Uniqueness of the Approach

The key distinction of our approach is its **fully streaming-first design on Pathway**, where streaming functions as the execution layer for feature computation, learning, decisioning, and auditability rather than only data transport. Most existing open-source and commercial fraud systems process streams with largely stateless models and depend on **periodic batch retraining** to handle drift [4]. In contrast, our platform uses **incremental online learning** that adapts continuously to evolving fraud patterns and customer behavior without retraining downtime.

While conventional systems treat fraud detection, explainability, reporting, and customer engagement as siloed components, our system unifies **real-time ingestion, stateful feature computation, rule-based safeguards, ML inference, and human-in-the-loop feedback** within a single streaming pipeline. Explainability is integrated directly into the streaming loop, where an **LLM-grounded layer** generates regulator-ready summaries from the same live features and decisions used for inference. The same continuously updated customer state also supports event-driven targeting, unifying risk mitigation and engagement in one system.

In traditional banking, customer targeting relies heavily on static CRM snapshots and fixed sales mandates, leading to outreach based on outdated segments rather than real customer intent. Although some banks have improved personalization through targeted engagement efforts [7], these approaches remain largely campaign-driven. Our platform advances this by enabling a stream-native, eligibility-aware targeting pipeline where recommendations emerge from live transactions, portfolio updates, and continuously refreshed profiles. Combined with an intelligent caller agent, the system ensures outreach occurs only when customers are genuinely eligible and financially ready, shifting engagement from push-based selling to contextual, trust-preserving interaction that scales without manual pressure.

3. Problem Understanding & Use-Case Validation

This project aligns directly with the problem statement's focus on building **production-grade, streaming-native AI systems** that operate on continuously arriving data. The selected use cases, *real-time fraud detection* and *customer targeting in banking*, are inherently time-sensitive and unsuitable for batch-oriented or periodically refreshed pipelines. Digital payment frauds such as card-not-present transactions occur within seconds, leaving no tolerance for delayed detection, while customer eligibility and intent evolve dynamically with every transaction, salary credit, or portfolio change.

A streaming-native setup is therefore essential. Live tables enable continuous aggregation of recent user behavior such as transaction velocity and spending anomalies, real-time indexes support low-latency contextual feature retrieval, and continuously updated metrics allow models to adapt without retraining downtime. Using streaming ingestion and stateful processing, the system performs in-flight fraud scoring, instant alert generation, and real-time customer segmentation, capabilities that are fundamentally incompatible with batch-based architectures.

The problem is both realistic and production-relevant, as reinforced by regulatory and industry signals. Financial regulators including the *Reserve Bank of India* [1] and the *European Union (GDPR)* [2] mandate explainability, auditability, and timely controls for automated decision systems. Industry reports consistently indicate a year-over-year rise in digital fraud [3], resulting in substantial financial and reputational losses for banks.

At the same time, banks are increasingly investing in real-time personalization platforms to improve cross-sell and retention outcomes [7, 8]. However, traditional CRM systems still operate on static snapshots, leading to mistimed outreach that fails to reflect customers' current financial readiness. Product eligibility and intent change continuously with income flows, repayment behavior, and short-term cash-flow volatility, making customer targeting an inherently streaming problem.

Our approach mirrors this reality using only first-party banking signals such as income credits, balances, credit limits, EMIs, spending patterns, and derived metrics already available within core banking and CRM systems. For fraud detection, we validate the system using a publicly available credit card fraud dataset [5], reflecting a well-studied and production-relevant problem structure. Together, these choices ensure the proposed solution is technically feasible, realistic, and representative of real-world banking systems that demand a streaming-first design.

4. Solution Overview

4.1 Design and Streaming-First Architecture

The system is organized into two major pipelines, both designed to operate entirely in streaming mode. At its core, the architecture leverages **Pathway's streaming primitives** such as connectors, live tables, reducers, and persistent state to ensure all decisions are made on continuously updated data rather than periodic snapshots.

Incoming transactions are ingested through streaming connectors such as NATS and materialized as *live tables*. Stateful reducers maintain rolling aggregates and recent behavioral features, including transaction velocity and spending deviations. **Deterministic rule engines** operate directly on these live features to enforce regulatory constraints, fraud checks, and eligibility guardrails, providing fast and predictable control for high-risk actions.

On top of this layer, **incremental ML models** handle pattern recognition and anomaly detection. The fraud detection pipeline employs an online Online tree-based models (Half-Space tree) that adapts continuously as feedback arrives, while the customer targeting pipeline uses Gaussian Mixture–based clustering over live embeddings to reflect current financial intent rather than static CRM segments. Customer state is maintained using an in-memory Redis-backed feature store, which can be directly mapped to a bank's internal CRM in production settings.

LLMs are integrated as **grounded explainability components** rather than standalone decision-makers. Using Pathway's LLM xPack, explanations and reports are generated from the same live features, model outputs, and business rules used during inference, ensuring transparency and auditability. Streaming execution is essential because both fraud and customer intent evolve over time, where milliseconds of delay can cause financial loss or missed opportunities. Live indexes and incremental updates allow features and models to evolve in sync with incoming events, a capability that batch-oriented architectures fundamentally lack.

4.2 Agent Flow and Decision Orchestration

We have mostly kept the use of agents and tools for the latter part of both our pipelines. This is because, our decision making requires fast reactivity and low latency. Hence, we have used agents for the explainability and report generation layer.

These events may trigger downstream actions, including alert manager notification or outreach via contact agents. Feedback is sent using dashboards for alert managers and we have used this human-in-the-loop pipeline for injecting feedback and updating our model.

For our demo purpose, we also have integrated the Vapi API [9] for making the call to concerned customer using the contact agent. The main purpose of the contact agent is to take in the listing of customers to be targeted (provided by our model) and the **live indexed** bank offer storage and make a call to the concerned customer. The contact agent is also in charge of generating *on-demand* and *batched* reports regarding the explanation behind why the particular customers were targeted.

5. System Architecture

5.1 Fraud detection pipeline

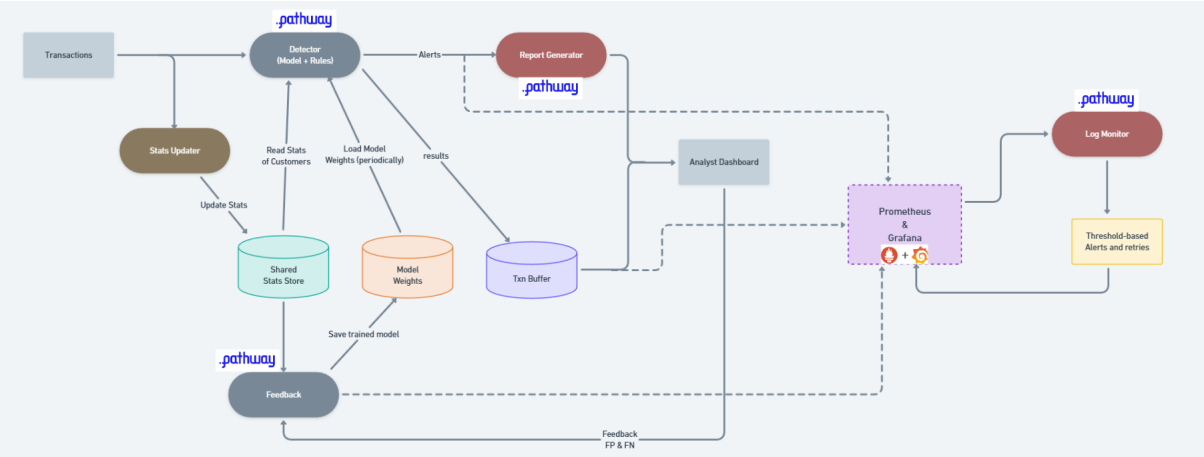

Figure 1: Architecture of the Fraud Detection Pipeline

The fraud detection pipeline is built as a fully streaming, event-driven system using Pathway. Incoming credit card transactions are published to *NATS* topics and ingested through Pathway's native connector. The stream is processed by two parallel Pathway processors: a **decision processor** that combines configurable rule-based checks with ML-based fraud scoring, and a **feature update processor** that maintains live customer features in a shared *Redis*-backed store (e.g., transaction velocity and spending patterns). The decision processor reads directly from this shared feature store to ensure consistent, low-latency inference.

Flagged transactions are published to an *alerts* topic and persisted in a lightweight *SQLite* transaction buffer for traceability. A downstream report-generation processor consumes the alerts stream to generate structured, human-readable fraud reports using bank-defined rules, which are surfaced on a real-time analyst dashboard.

Investigator feedback on false positives and false negatives is streamed back through a *feedback* topic. A dedicated feedback processor periodically updates the model parameters, which are reloaded by the decision processor without interrupting live processing. This tightly integrated streaming architecture enables low-latency detection, explainability, and continuous adaptation within a single execution framework.

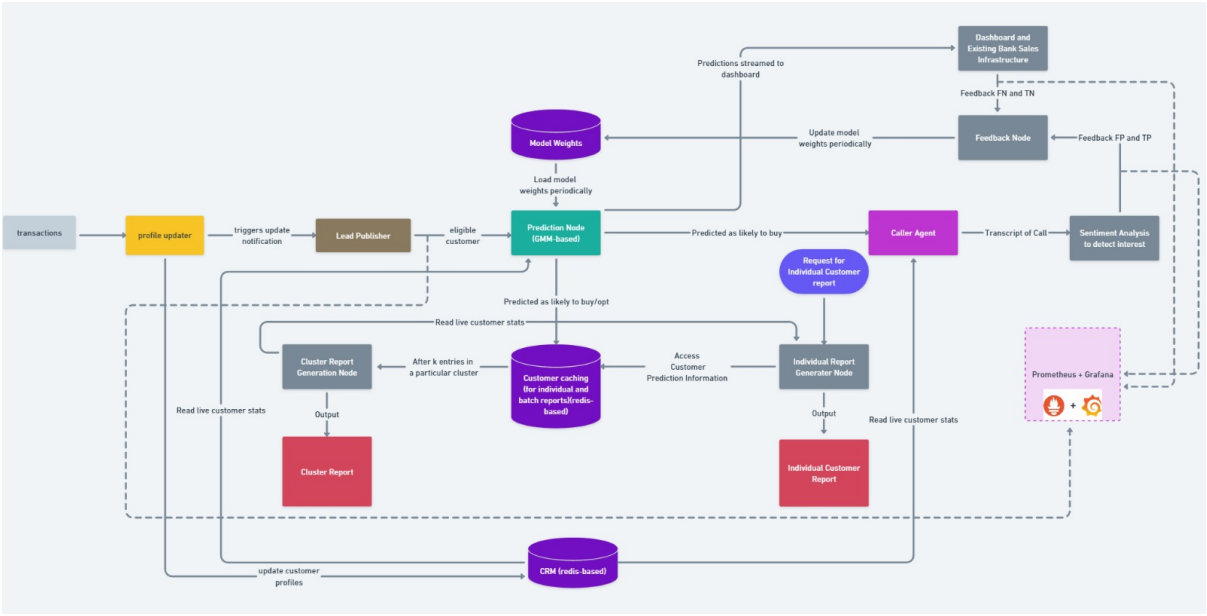

Figure 2: Architecture of the Customer Targetting Pipeline

5.2 Customer Targetting pipeline

The targeted-calling pipeline is implemented as a fully streaming, event-driven system built on Pathway. Customer transactions are streamed through NATS and ingested via Pathway connectors, then processed by a profile-update node that incrementally maintains customer features such as balances, spending signals, DTI, savings rate, and credit-score changes inside a live CRM table. Each update triggers a lead-publisher processor that determines whether the customer should be evaluated for the selected financial product.

Eligible profiles are sent to a prediction processor running a hybrid GMM-based intent model, which is periodically refreshed and supports online updates. The model outputs purchase-intent scores and cluster IDs, and forwards high-intent customers to a dedicated topic powering the caller-agent module. Using live Pathway-indexed CRM state and active offers, the agent generates personalized outreach through the Vapi API. Call transcripts are analyzed by a downstream sentiment processor, producing structured feedback signals.

Feedback from sentiment analysis and sales review is consumed by an update processor that applies incremental corrections to the model weights, which are reloaded by the prediction node without disrupting streaming inference. Per-customer predictions are cached in a Redis-backed store to support explainability, and cluster-level reports are generated when sufficient evidence accumulates.

Operational telemetry including throughput, NATS traffic, latency, and memory usage is exported to Prometheus and visualized on Grafana, while a Pathway-based log monitor issues alerts and retries based on threshold conditions. The result is a cohesive, low-latency system where real-time computation, continuous learning, explainability, and agent-driven actions operate within a single streaming architecture.

6. Results and Metrics

6.1 Fraud Detection Pipeline

For the purpose of testing and demonstrating this pipeline, we used the credit card frauds dataset of 1.9M transactions. For this, we first pre-trained the model on a subset (12K) of the labelled dataset. We then streamed the main transactions (without target labels) from this csv file and then artificially injected the feedback stream as a delay of the transaction stream. We then observed the metrics as well as latency in our grafana dashboard and got running stable F1 scores of 87 ± 2 which nearly matches the static[6] F1 scores. We also have shown the real-time logged metrics in the form of tables and graph in the following figure.

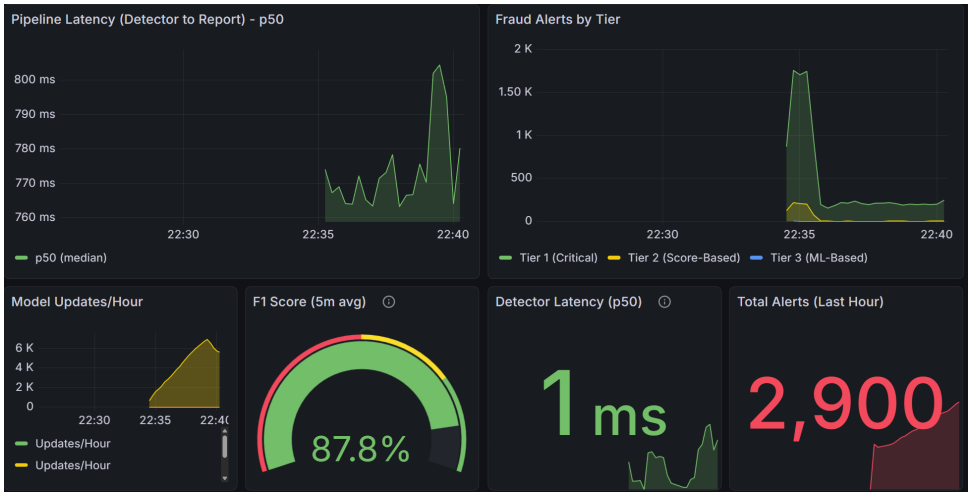

Figure 3: Example of our live-metrics reported using grafana.

6.2 Car Loan Propensity Streaming Pipeline

To evaluate the streaming customer–propensity engine, we generated a synthetic multi-product dataset with 10,000 customers for offline training and 2,500 customers for online simulation. The system follows a hybrid architecture, where a static "oracle" neural network classifier is trained on the full feature space, while the streaming engine uses a dynamically updating **Gaussian Mixture–based latent state tracker** to adapt to real-time behavioral drift.

The oracle model achieved an accuracy of 0.88, with precision 0.89 and recall 0.97 on a held-out validation set. During online simulation, this oracle provided delayed supervisory feedback, mimicking real-world scenarios where customer responses are observed after an interaction. For streaming evaluation, 380K timestamped transactions spanning four months were processed. Customer features were updated incrementally, and the streaming model was triggered every 15 transactions per customer to evaluate cluster membership and generate real-time predictions.

Over 24,190 streaming update cycles, the adaptive model converged to a stable accuracy of 0.8334, with precision 0.8649 and recall 0.8738. The F1 score closely approached the oracle benchmark, indicating successful adaptation without full retraining. As expected, early metric volatility due to sparse clusters gradually diminished as the GMM specialized around stable behavioral patterns.

Detailed results, including streaming accuracy curves, confusion matrices, and cluster evolution visualizations, are presented in the following figures, illustrating the model's ability to track customer intent and behavioral transitions in real time.

7. Implementation Details and Production Bar

7.1 Streaming Ingestion, Live Indexing, and Resilience

The system is implemented as a fully streaming application where all data ingestion, feature computation, and decision-making operate on continuously arriving events. Banking events such as transactions and customer updates are ingested through **NATS-based streaming connectors** and immediately materialized into **Pathway live tables**.

For fraud detection demonstration, we have used the existing static dataset mentioned above, and streamed it using an artificial publisher into our pipeline. For simulating feedback we simply streamed the same dataset but with certain lag so that the model corrects its mistakes. Although in the final deployment, this will be done manually by a fraud analyst, which will decide from our results storage which frauds are not detected (false negatives) and which ones are unnecessarily flagged (false positives). For customer targetting demonstration we followed the same procedure as above but with the synthetically generated dataset described before.

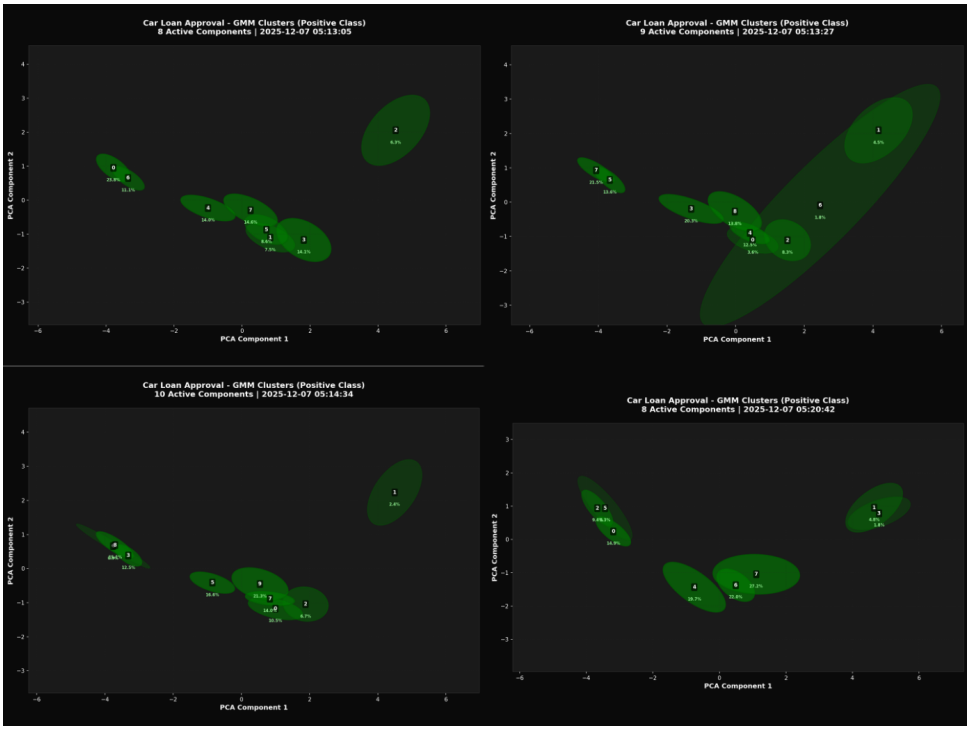

Figure 4: Visualisation of the changing decision boundary of our GMM model under addition of cluster (1st row) and changing positions (2nd row).

To support low-latency inference, frequently accessed and updated user-related context is stored in an in-memory **redis** instance, allowing fast queries during decision execution. All processing stages are designed to be idempotent, ensuring safe replays in case of transient failures. Deterministic rule-based checks also act as fallbacks whenever model inference is unavailable or uncertain. This guarantees that high-risk decisions default to conservative, rule-enforced outcomes rather than failing silently. **Live Indexing** is used to store the bank's offers and metadata related to compliancies. This is used when generating reports or contacting the eligible customer, so that the agent always uses the most up-to-date and relevant information for such. In addition to data-level resilience, we integrated **log-aware control using Pathway**. Runtime logs emitted by model pipelines and processors are continuously streamed back into Pathway tables, where thresholds on error rates, latency spikes, or abnormal event patterns are monitored. When these thresholds are breached, the system can automatically trigger alerts or restart affected components, enabling self-healing behavior under degradation scenarios.

7.2 Production Practices: Observability, Testing, and Deployment

To ensure production-grade observability, we implemented a comprehensive **logging and monitoring stack** using **Prometheus and Grafana**. Core operational metrics, including end-to-end latency, event throughput, alert rates, model decisions, and error counts, are continuously tracked and visualized on live dashboards, enabling rapid detection of performance bottlenecks and anomalous behavior under streaming load.

The entire system is **fully containerized using Docker**, with streaming connectors, Pathway processors, ML inference modules, and monitoring services deployed as isolated, reproducible components. A single startup configuration launches the complete pipeline, including ingestion, processing, and observability, enabling one-click local or on-prem deployment that mirrors real production environments.

To validate stability, we conducted basic and end-to-end tests such as injection tests for system reactivity, concept drift scenarios, and pipeline fallback and restart under load. Additional guardrails include schema validation at ingestion, conservative rule overrides for uncertain predictions, and bounded update rates for online learning. Together, these measures ensure the system is robust, observable, and deployable as a realistic

production-ready streaming AI platform.

8. Challenges Faced and Solutions

One of the primary challenges was **designing low-latency, stateful streaming pipelines** that could reliably join heterogeneous event streams such as transactions, login events, and customer profiles. We often struggle with maintaining consistency under out-of-order or delayed events; this was addressed by using Pathway's stateful tables and time-aware joins, ensuring that feature computation remained stable and reproducible.

Another key difficulty was implementing **online learning without model instability**. Incremental updates can easily introduce drift or oscillations when feedback is sparse or noisy. We mitigated this by carefully bounding update rates and incorporating lightweight rule-based guardrails around learned models.

Finally, achieving **explainability in real time** is a common bottleneck in production systems. Instead of relying on post-hoc explanations, we integrated an LLM-based explanation layer directly into the streaming loop, grounding outputs in live features and stored rules. This ensured that every decision remained traceable, human-interpretable, and compliant with regulatory expectations.

9. Lessons Learned

A key insight from this project was that **building streaming AI systems is primarily a systems engineering challenge**, not just a modeling problem. We observed that correctness, latency, and explainability depend more on data flow design, state management, and observability than on model complexity alone. Integrating monitoring and log-driven control early significantly improved system stability and debuggability. Finally, grounding LLM outputs strictly in live data and deterministic rules proved essential for maintaining trust, auditability, and production suitability.

10. Future Plan

BDH introduces a more brain-inspired way of reasoning over streaming data, allowing models to understand how behavior unfolds over time instead of treating each event as an isolated snapshot. In our system, BDH could naturally sit after the feature store and before final decision-making, where it would continuously reason over a customer's transaction history, behavioral changes, and evolving context.

This makes it especially useful for catching subtle fraud patterns or gradual shifts in customer intent that simpler rule-based checks or short-term features often miss. At the same time, BDH remains suitable for real-time use: its sparse and interpretable activations help preserve transparency, low latency, and regulatory compliance, while enabling more powerful long-term reasoning over live data streams.

11. Conclusion

This project delivers a production-grade, streaming-native AI platform that unifies real-time fraud detection and customer targeting within a single, low-latency system built on Pathway. By combining deterministic rules, incremental machine learning, and LLM-grounded explainability over live data streams, the platform demonstrates how banks can detect fraud in-flight, maintain continuously updated customer profiles, and generate regulator-ready insights without batch dependencies. The implementation emphasizes practical deployability through full containerization, robust observability, and self-healing mechanisms, making the system both technically sound and operationally realistic. Overall, the work validates that streaming-first architectures are not only essential but also achievable for high-stakes financial AI systems operating under strict latency and compliance constraints.

12. Appendix

References

[1] Reserve Bank of India, "Draft Guidelines on Regulatory Principles for Management of Model Risks in Credit," 2024. Available: https://www.fidcindia.org.in/wp-content/uploads/2024/08/RBI-DRAFT-MANAGEMENT-OF-MODEL-RISK-IN-CREDIT-05-08-24.pdf.

[2] European Union, "General Data Protection Regulation (GDPR) — Articles 13–22: Automated Decision-Making, Transparency, and Auditability." Available: https://eur-lex.europa.eu/eli/reg/2016/679/oj.

[3] eMarketer, "Card-not-present fraud in payments," *Insider Intelligence – eMarketer*, 2024. Available: https://www.emarketer.com/content/card-not-present-fraud-payment.

[4] A. Gupta et al., "Fraud Detection in Banking Using Real-Time Data Stream Analytics and AI For Improved Security and Transaction Monitoring," *Propulsion Tech Journal*, vol. 12, no. 3, 2024. Available: https://www.propulsiontechjournal.com/index.php/journal/article/download/9543/5822/16152.

[5] Kartik K., "Credit Card Transactions Fraud Detection Dataset (Synthetic)," Kaggle. Available: https://www.kaggle.com/datasets/kartik2112/fraud-detection?select=fraudTrain.csv.

[6] Muneeb Ullah, "Static Model statistics trained on Credit Card Fraud dataset," Kaggle. Available: https://www.kaggle.com/code/muneebullahmuneeb/fraudsight-xgboost-powered-fraud-detection.

[7] Contentree, "Axis Bank achieves a 27% boost in conversions by enhancing user experiences with CleverTap," 2024. Available: https://www.contentree.com/caseStudy/axis-bank-achieves-a-remarkable-27-boost-in-conversions-by-enhancing-user-experiences-with-clev
427778.

[8] Krungsri Research, "Hyper-personalization: Giving banks AI-powered insight into their customers," 2024. Available: https://www.krungsri.com/en/research/research-intelligence/ai-hyper-personalization-2024.

[9] Vapi, "Quickstart," Vapi Documentation, 2025. Available: https://docs.vapi.ai/quickstart.